

React **children** Concept — Complete Notes

1. **children** কী?

React-এ কোনো component-কে এভাবে লিখলে:

```
<Card>  
  <Profile />  
</Card>
```

<Card> এবং **</Card>**-এর মাঝখানে যা আছে:

```
<Profile />
```

সেটাই **Card** component-এর **children prop**।

সহজভাবে:

children হলো কোনো **component**-এর **opening** এবং **closing tag**-এর মাঝখানে থাকা **content**।

2. **children** আসলে একটি **prop**

আমরা সাধারণত props এভাবে পাঠাই:

```
<User name="Farhan" age={25} />
```

এখানে:

```
name  
age
```

হলো normal props।

একইভাবে:

```
<Card>
  <Profile />
</Card>
```

এখানে React automatically একটি prop তৈরি করে:

children: <Profile />

তাই component-এ আমরা লিখতে পারি:

```
function Card({ children }) {
  return (
    <div>
      {children}
    </div>
  );
}
```

এখানে:

{children}

মানে:

Parent component থেকে যে content **children** হিসেবে এসেছে, সেটা এখানে render করো।

3. **children** শুধু **component** হতে হবে এমন নয়

children হিসেবে বিভিন্ন ধরনের content পাঠানো যায়।

Text

```
<Card>
  Hello
</Card>
```

এখানে:

```
children = "Hello"
```

HTML/JSX element

```
<Card>  
  <h1>Hello</h1>  
</Card>
```

এখানে:

```
children = <h1>Hello</h1>
```

অন্য component

```
<Card>  
  <Profile />  
</Card>
```

এখানে:

```
children = <Profile />
```

একাধিক element/component

```
<Card>  
  <h1>Hello</h1>  
  <Profile />  
  <button>Click</button>  
</Card>
```

এখানে সবগুলোই **Card**-এর children:

```
Card  
├── children  
│   ├── h1  
│   ├── Profile  
│   └── button
```

4. **children** ব্যবহার করার মূল সুবিধা

ধরো:

```
function Card({ children }) {  
  return (  
    <div className="card">  
      {children}  
    </div>  
  );  
}
```

এখন **Card** নিজে জানে না তার ভিতরে কী থাকবে।

যে ব্যবহার করবে, সে ঠিক করবে:

```
<Card>  
  <Profile />  
</Card>
```

অথবা:

```
<Card>  
  <Product />  
</Card>
```

অথবা:

```
<Card>  
  <LoginForm />  
</Card>
```

অর্থাৎ একই **Card** component বিভিন্ন content-এর জন্য ব্যবহার করা যায়।

মূল idea:

Card = container/wrapper

children = ওই container-এর ভিতরের content

5. তাহলে সরাসরি **component import** করে ব্যবহার করলেই তো হয়?

হ্যাঁ, অবশ্যই হয়।

ধরো:

```
import Profile from "../Profile";
```

```
function Card() {  
  return (  
    <div>  
      <Profile />  
    </div>  
  );  
}
```

এটা সম্পূর্ণ valid React code।

এখানে **children** ব্যবহার করা হয়নি।

এখন:

```
<Card />
```

করলেই **Card**-এর ভিতরে **Profile** render হবে।

6. Direct import বনাম **children**

Direct import

```
import Profile from "../Profile";
```

```
function Card() {  
  return (  
    <div>  
      <Profile />  
    </div>  
  );  
}
```

এখানে **Card** নিজেই ঠিক করে দিয়েছে:

আমার ভিতরে **Profile** থাকবে।

অর্থাৎ relationship fixed:

Card
└─ Profile

children ব্যবহার করলে

```
function Card({ children }) {  
  return (  
    <div>  
      {children}  
    </div>  
  );  
}
```

এখন **Card** জানে না ভিতরে কী থাকবে।

ব্যবহারকারী decide করবে:

```
<Card>  
  <Profile />  
</Card>
```

অথবা:

```
<Card>  
  <Product />  
</Card>
```

অথবা:

```
<Card>  
  <button>Click</button>  
</Card>
```

7. Next.js layout-এ children

এখানেই Next.js-এর গুরুত্বপূর্ণ অংশ।

Next.js-এর layout:

```
export default function RootLayout({children,}: {children:
React.ReactNode;})
{
  return (
    <html>
      <body>
        <Navbar />

        {children}

        <Footer />
      </body>
    </html>
  );
}
```

এখানে আমরা manually লিখছি না:

```
<RootLayout>
  <HomePage />
</RootLayout>
```

তবুও children পাওয়া যাচ্ছে।

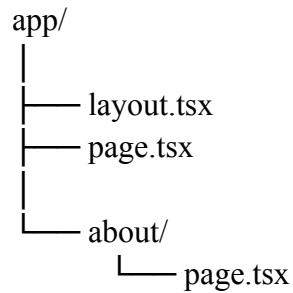
কেন?

কারণ Next.js routing system automatically current route-এর page-কে layout-এর children হিসেবে দেয়।

অর্থাৎ এখানে children-এর value Next.js automatically provide করছে।

8. Next.js-এ current page কীভাবে **children** হয়?

ধরো folder structure:

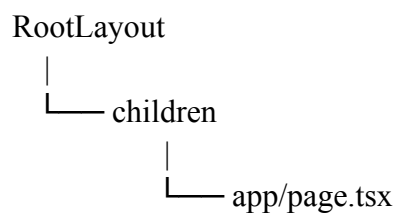


তুমি / **route**-এ গেলে

Next.js automatically বুঝবে:

Current page = app/page.tsx

তারপর conceptually:

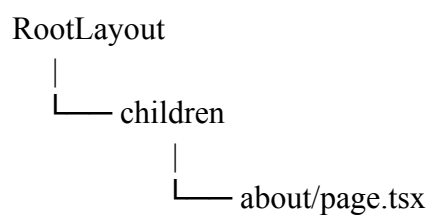


তুমি / **about** route-এ গেলে

Next.js automatically বুঝবে:

Current page = about/page.tsx

তারপর:



তাই:

```
{children}
```

এর অর্থ এখানে:

বর্তমান **route**-এর **page/content** এখানে **render** করো।

9. React **children** বনাম Next.js **layout children**

দুটো একই basic React concept ব্যবহার করে, কিন্তু content কে পাঠাচ্ছে সেটা আলাদা।

Normal React

```
<Card>  
  <Profile />  
</Card>
```

এখানে:

Parent/developer



children



Card

অর্থাৎ তুমি manually content পাঠাচ্ছে।

Next.js Layout

```
function RootLayout({ children }) {  
  ...  
}
```

এখানে:

Next.js routing system



children



RootLayout

অর্থাৎ Next.js automatically current route-এর page/content পাঠাচ্ছে।

10. Component hierarchy / parent-child relationship

ধরো:

Technologies
└── TechnologyItem

এর অর্থ:

Technologies
↓
TechnologyItem

TechnologyItem হচ্ছে **Technologies**-এর child component হিসেবে ব্যবহৃত হচ্ছে।

যেমন:

```
function Technologies() {  
  return (  
    <div>  
      <TechnologyItem />  
    </div>  
  );  
}
```

এখানে:

Technologies
└── TechnologyItem

তাহলে অন্য **component** কি **TechnologyItem** call করতে পারবে?

Technically: হ্যাঁ।

যদি **TechnologyItem** export করা থাকে:

```
import TechnologyItem from "../TechnologyItem";
```

```
function About() {  
  return (  
    <div>  
      <TechnologyItem />  
    </div>  
  );  
}
```

React-এর দিক থেকে এতে কোনো error নেই।

তখন hierarchy হয়ে যাবে:

```
Technologies —→ TechnologyItem  
About      —→ TechnologyItem
```

এটা technically valid।

But,

যদি **TechnologyItem** এমনভাবে design করা হয় যে সেটা শুধু **Technologies** component-এর internal অংশ, তাহলে অন্য component থেকে direct use না করাই ভালো।

ধরো:

```
Technologies  
└── TechnologyItem
```

এখানে তুমি ধরে নিয়েছো:

TechnologyItem শুধু Technologies-এর জন্য।

তখন অন্য component থেকে:

```
About  
└── TechnologyItem
```

ব্যবহার করলে architecture-এর intended responsibility বদলে যাচ্ছে।

Reusable component multiple time use kora possible:

ধরো:

Button
Card
Modal
Navbar

এগুলো সাধারণত reusable।

একই **Button** বিভিন্ন component থেকে ব্যবহার করা স্বাভাবিক:

Home
└─ Button

About
└─ Button

Contact
└─ Button

Parent-specific component হলে

ধরো:

Technologies
└─ TechnologyItem

যদি **TechnologyItem** শুধু **Technologies**-এর জন্য তৈরি করা হয়, তাহলে এটাকে parent-specific/internal component হিসেবে রাখা যায়।

Conceptually:

Technologies
└─ TechnologyItem

এবং অন্য component-এ **direct use** না করাই **cleaner architecture**।